UNIVERSITÀ DI PISA

Dipartimento di Matematica

Analisi dei dati

Compito Finale : EigenWorms

insegnanti:

MARCO ROMITO

AIKATERINI PAPAGIANNOULI

Studente:

LOUIS URBIN

ANNO ACCADEMICO 2024/2025

Indice

Introduction

This dataset, originally due to Brown et al., is composed of different motions of worms on an agar plate. Each motion is decomposed into six dimensions. In fact, it has been shown that the space of shapes that *Caenorhabditis elegans* adopts on an agar plate can be represented as combinations of six base shapes, called eigenworms.

Thus, for each experiment in the dataset, we have six time series—one for each dimension—representing the projection of the worm's motion onto the corresponding eigenworm dimension.

Our objective is twofold:

1. **Classification.** We apply different methods: Random Forest, SVM with selected features, LSTM, and GRU models with varying depth, all aimed at classifying the time series. We then explore an alternative idea: instead of feeding our LSTM/GRU models with the six dimensions jointly (six time series as input), we try processing each dimension independently.

   Next, we attempt to reduce the length of the time series using methods like decomposition and downsampling. Finally, we adopt a very different approach: treating the six time series as multichannel images and applying CNNs to detect patterns.

2. **Interpretability.** In this second task, we aim to develop methods that help us understand which characteristics of the motion are most relevant to the output of our best classification model. We explore two different approaches: perturbation-based analysis and backpropagation-based interpretation techniques.

For all our tests, we will mainly use accuracy as the evaluation metric, to determine whether the model correctly predicts the worm class. We will also include confusion matrices to assess whether certain types of worms are better classified than others.

Capitolo 1

Data Exploration and Preprocessing

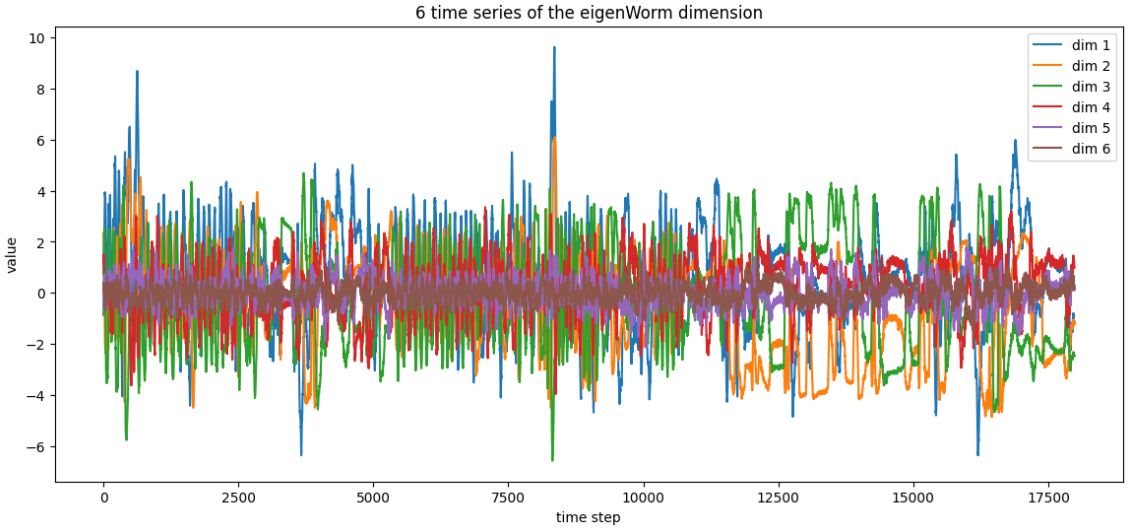
We began by exploring the dataset files, which are provided in the `.arff` format. These include separate training and testing sets. Our first task was to decode and convert these files into `pandas` DataFrames, which allows for easier manipulation and analysis in Python.

The data consists of multivariate time series, and our goal at this stage was to understand how these time series were encoded in the `.arff` structure. Upon inspection, we observed that each sample is composed of a multivariate signal with six components—corresponding to the six eigenworm dimensions—stored in a custom format. Fortunately, there appear to be no missing values in the dataset.

We merged the train and test sets into a single DataFrame, and later recreated a training/testing split using `train_test_split` from `sklearn`. However, upon calling `.info()` on the DataFrame, we noticed that only two columns were detected, with each showing 128 non-null entries. One of these columns is the `target` column, which contains five class labels: `b'1'`, `b'2'`, `b'3'`, `b'4'`, and `b'5'`. Among these, four correspond to mutant worms and one represents the wild-type.

To better understand the structure of the input data, we attempted to visualize the time series. Each entry in the `eigenWormMultivariate_attribute` column is in fact a NumPy array of shape `(6, 17984)`, where 6 refers to the six eigenworm dimensions, and each row contains a sequence of values stored as a `numpy.void` type—essentially a serialized time series.

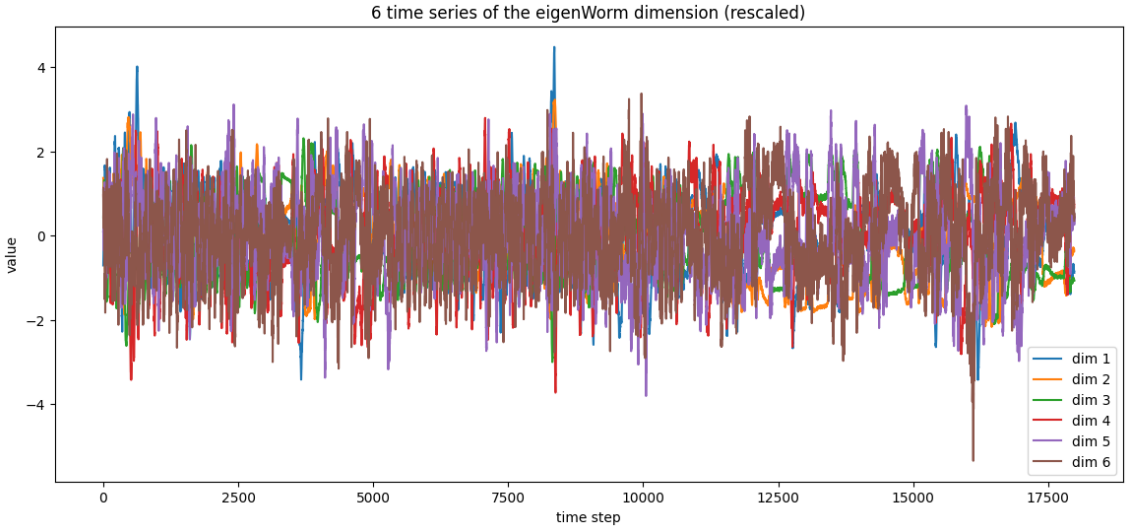
We began by plotting the time series from the first example.

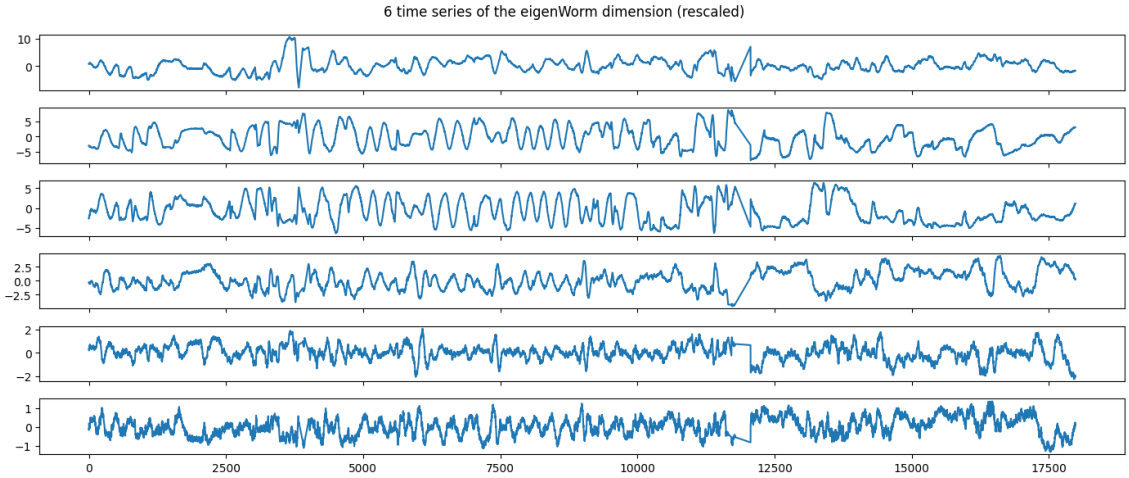


Upon visual inspection, we noticed that certain dimensions appear to have consistently larger norms than others. This could potentially bias our models toward those dimensions. Therefore, it is worth considering whether rescaling is necessary—just as it is standard practice to rescale features before applying most classification algorithms.

To investigate further, we plotted time series from examples of all five classes. We observed that the amplitude range for each dimension was similar across all classes. Thus, the amplitude itself does not seem to be a discriminative feature, and we can safely rescale our data without losing relevant class information.

To standardize the data, we applied a transformation to ensure each dimension has zero mean and unit variance.





For easier model input formatting, we stored the complete dataset as a 3D NumPy array with shape `(n_samples, n_timesteps, n_features)`.

Another important observation was that the dataset samples were sorted by class. If left unshuffled, this could severely bias the training process, as the model might only see certain classes during specific training phases. Therefore, we ensured that the data was properly shuffled before training.

We also noticed a degree of class imbalance. In the training set, the distribution of classes was approximately: `[88, 35, 27, 35, 22]`, with class 0 representing close to 40% of the data. This imbalance could lead to deceptively high accuracy scores if the model simply predicts the majority class. To mitigate this, we used class weighting during model training.

Capitolo 2

Classification

We first focus on the primary classification task.

2.1 Classical Feature-Based Classification

Initially, we adopt a classical approach by manually extracting features (for each dimension: mean, standard deviation, min, max, median, energy, dominant frequency, and autocorrelation at lag 1) and using standard classifiers such as Random Forest and SVM. The goal is to establish a baseline. In addition to accuracy, we examine the confusion matrix to assess if some worm types are more easily classified.

Using an SVM yields an accuracy of 0.42. This classifier performs poorly as it tends to predict the majority class. We tested different kernels (sigmoid, RBF, polynomial), but without success.

	precision	recall	f1-score	support
1	0.91	0.95	0.93	22
2	1.00	0.89	0.94	9
3	1.00	1.00	1.00	8
4	0.90	0.90	0.90	10
5	0.67	0.67	0.67	3
accuracy			0.92	52
macro avg	0.90	0.88	0.89	52
weighted avg	0.92	0.92	0.92	52

Figura 2.1: SVM Performance

Using a Random Forest, results are much better, with an accuracy greater than 0.9.

	precision	recall	f1-score	support
1	0.91	0.95	0.93	22
2	1.00	0.89	0.94	9
3	1.00	1.00	1.00	8
4	0.90	0.90	0.90	10
5	0.67	0.67	0.67	3
accuracy			0.92	52
macro avg	0.90	0.88	0.89	52
weighted avg	0.92	0.92	0.92	52

Figura 2.2: Random Forest Performance

Although it's a bit related to the second task, we can already inspect which features are most important:

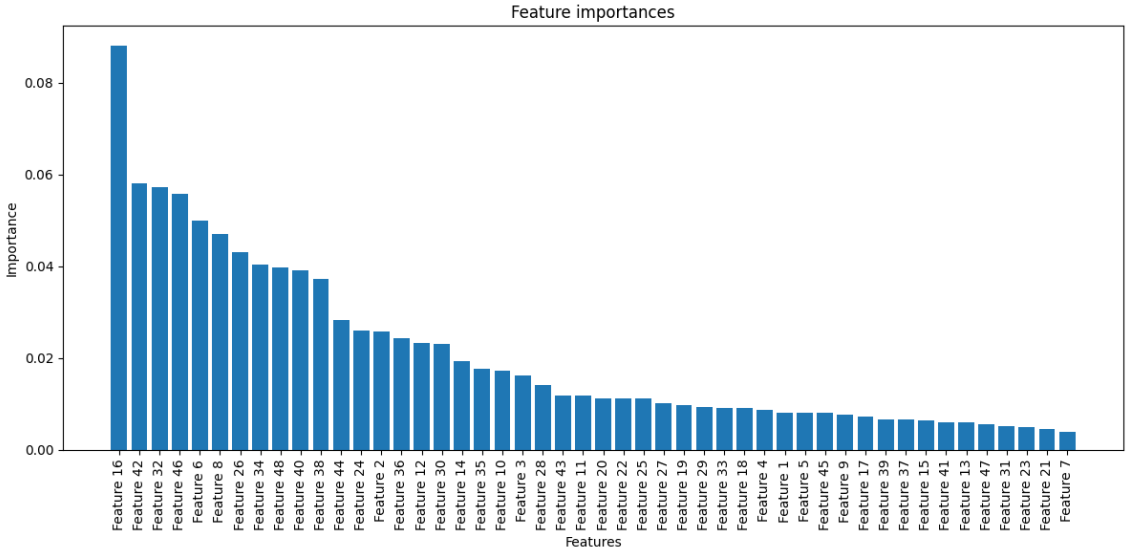


Figura 2.3: Random Forest Feature Importance

Feature 16, the autocorrelation of the second dimension at lag 1, stands out clearly. We will see later if further improvements can be achieved.

Note:

- From this point onward, and for the remainder of the deep learning section, we apply early stopping to prevent overfitting.
- Due to limited computational resources on Google Colab, we manually tuned hyperparameters instead of performing an exhaustive grid or random search.

2.2 Vanilla RNN

We experiment with a basic Recurrent Neural Network (RNN) to directly classify the time series. Despite tuning hyperparameters, the model fails to capture temporal patterns.

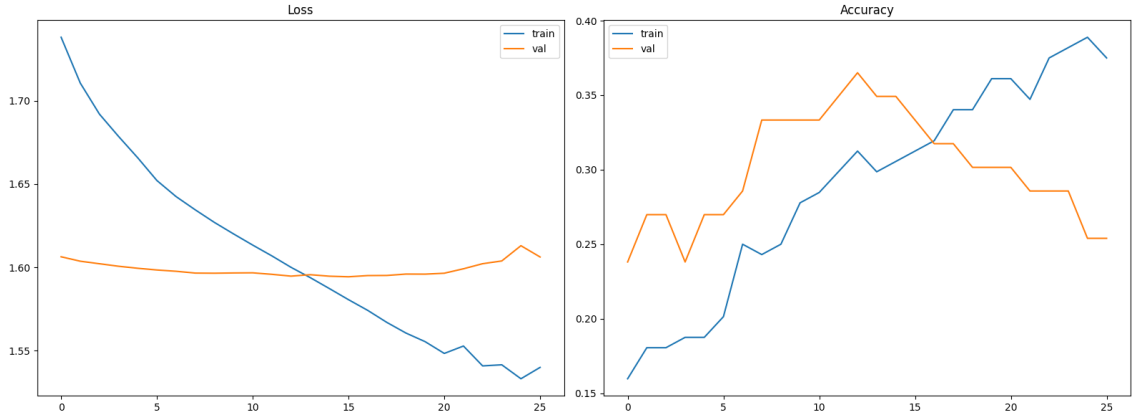


Figura 2.4: Vanilla RNN Performance

On the test set, the model achieves an accuracy of 0.28, worse than a naive classifier that predicts the majority class.

2.3 LSTM Networks

To better handle long- and short-term dependencies, we use Long Short-Term Memory (LSTM) networks.

A single LSTM layer yields an accuracy around 0.27, which is insufficient. The performance remains poor due to class imbalance (40% of instances belong to class 0).

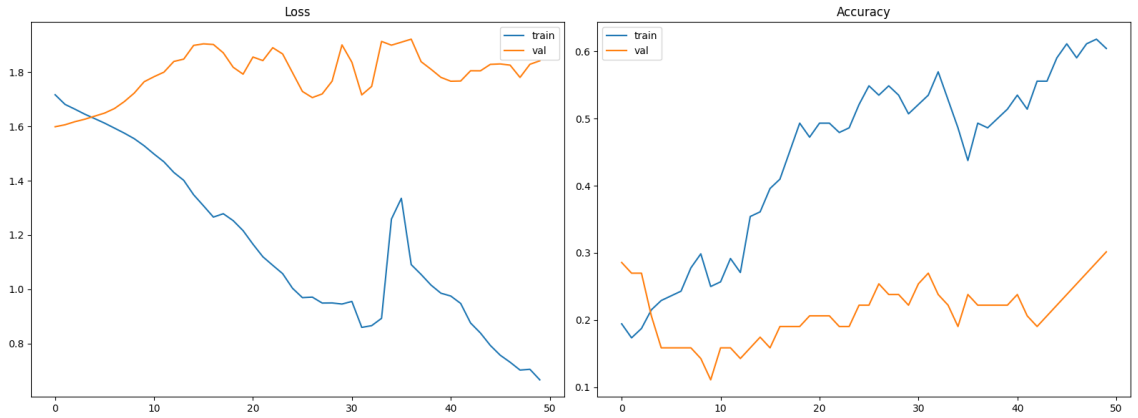


Figura 2.5: Single-Layer LSTM Performance

Using two LSTM layers (64 and 32 units) does not improve results.

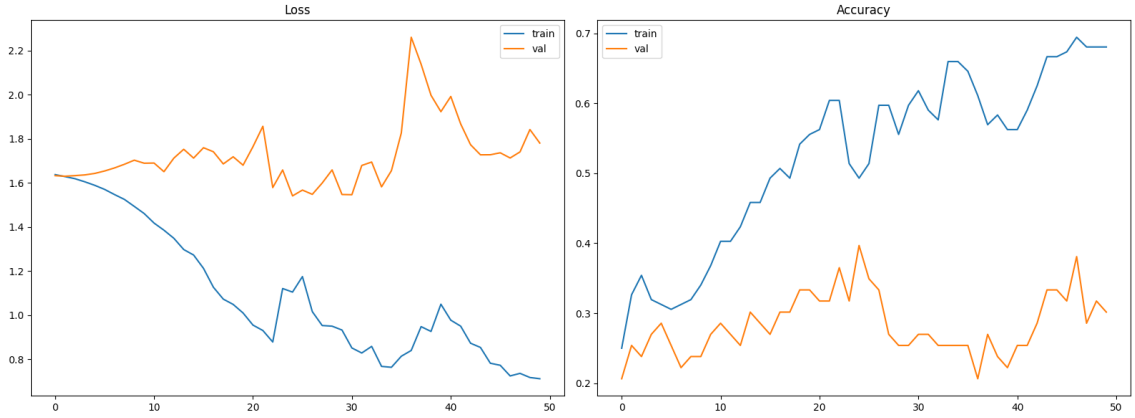


Figura 2.6: Two-Layer LSTM Performance

Increasing the number of epochs (from 100 to 1000) leads to significant overfitting. This suggests a need for dropout or L1/L2 regularization (applied in later models).

2.4 GRU Networks

To reduce overfitting while preserving temporal memory, we use Gated Recurrent Units (GRUs), which have fewer parameters than standard LSTM architectures.

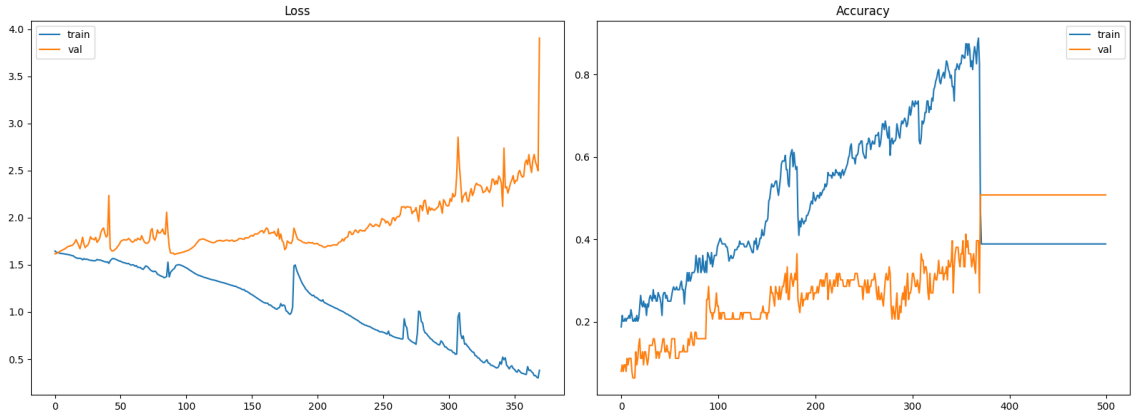


Figura 2.7: GRU Performance

On the test set, the GRU model reaches an accuracy of 0.42, which corresponds to the proportion of the majority class. As seen in the loss curve, the model begins to overfit after approximately 200 epochs.

We also experimented with an encoder-decoder architecture combining LSTM and GRU layers. However, this setup did not lead to better performance and the results remained unsatisfactory.

2.5 Independent Processing of Each Dimension

The initial strategy of feeding all six time series dimensions into a single RNN did not produce satisfactory results.

As an alternative, we consider treating each dimension independently. This involves training six separate models — one per dimension — and aggregating their predictions through majority voting. This approach also aligns with our secondary objective: assessing the individual relevance of each dimension.

Using a GRU with 64 units per dimension yields an overall test accuracy of 0.42, which simply reflects the proportion of the majority class.

Final accuracy after majority voting: 0.4231

	precision	recall	f1-score	support
1	0.42	1.00	0.59	22
2	0.00	0.00	0.00	9
3	0.00	0.00	0.00	8
4	0.00	0.00	0.00	10
5	0.00	0.00	0.00	3
accuracy			0.42	52
macro avg	0.08	0.20	0.12	52
weighted avg	0.18	0.42	0.25	52

Figura 2.8: Independent GRU Performance

Despite testing other architectures, this method does not yield better results. As it discards potential inter-dimensional correlations, it may hinder the model's capacity to capture the full temporal structure. We therefore return to approaches that preserve the joint processing of all dimensions to retain this valuable information.

Remark: Our time series are very long (17,984 time steps), which poses challenges for recurrent architectures such as LSTM and GRU that struggle to retain long-term information.

We explore four potential solutions:

1. Split each time series into 200-step segments, assigning each the full sequence label. This increases dataset size but may lose global context.

2. Downsample the time series to approximately 500 steps, assuming that capturing short-term patterns is sufficient.

3. Apply Convolutional Neural Networks (CNNs) to detect local temporal patterns.

4. Combine CNN layers with GRU or LSTM to capture local and long-term dependencies.

2.6 Segmentation Strategy

Segmenting time series into 200-step samples improves compatibility with LSTM architecture. Each segment inherits the label of the original sequence.

We classify new sequences by applying the same segmentation and aggregating segment predictions via majority voting. Class weights are used to mitigate imbalance.

We achieve an accuracy of 0.61 on the test set, which is significantly better than the previous results (excluding the Random Forest).

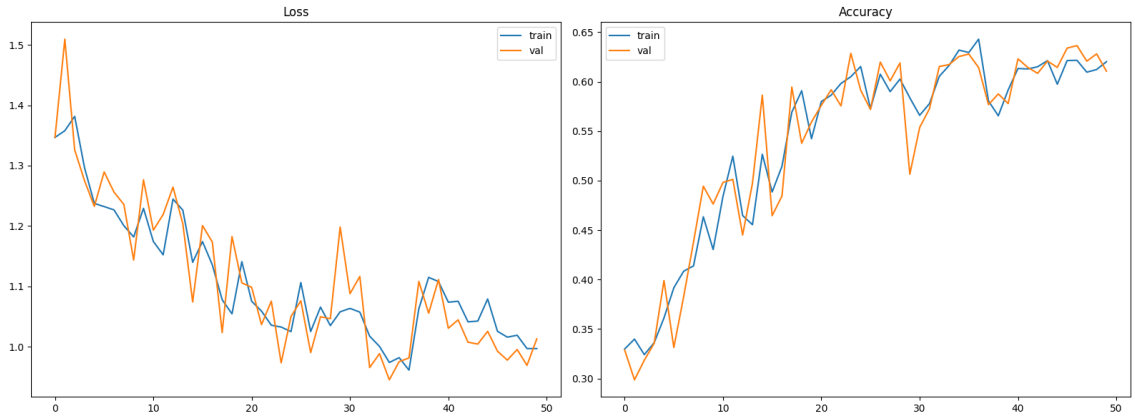


Figura 2.9: Segmented LSTM Classification

2.7 Downsampling Strategy

Downsampling reduces sequence length to about 500 steps, preserving the overall structure but lowering resolution.

We train LSTM models on this downsampled data using class weights.

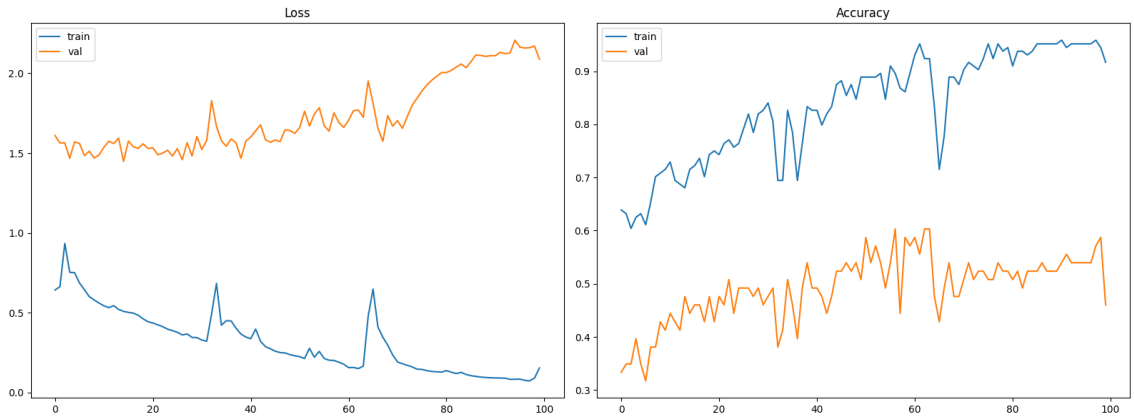


Figura 2.10: Downsampling Approach

The model achieves an accuracy of 0.36 — not sufficient.

2.8 CNN Approach

We use CNNs with kernel sizes of 5 and 15 to extract local temporal patterns (peaks, slopes, ...).

Surprisingly, CNNs achieve around 0.56 accuracy without using recurrence.

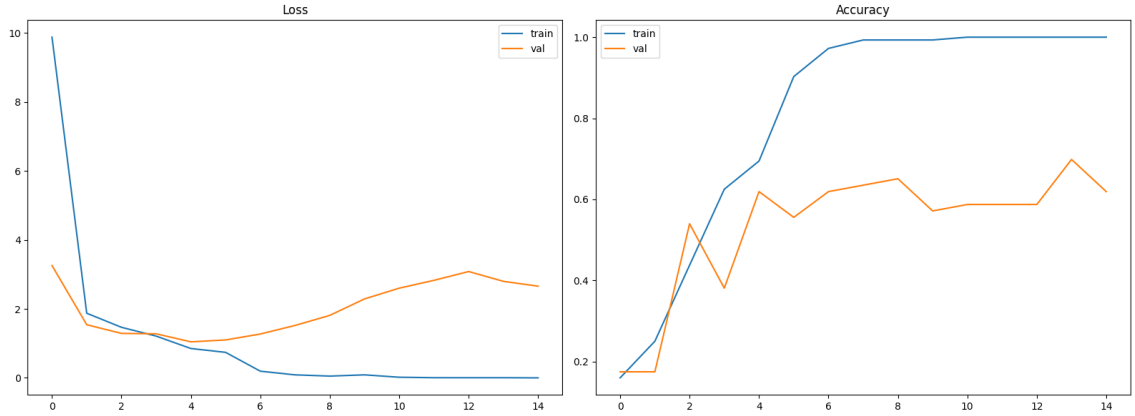


Figura 2.11: CNN-Based Classification

The CNN model has many parameters and is prone to overfitting. L1 (Lasso) and L2 (Ridge) regularization techniques did not yield better results — accuracy remains around 0.56.

The model is unstable: accuracy varies between 0.4 and 0.6 across different runs.

2.9 CNN + RNN Hybrid

To capture both local patterns and long-term dependencies, we combine CNN layers with GRU or LSTM.

The architecture includes two convolutional layers followed by max pooling (scale 2) and dropout.

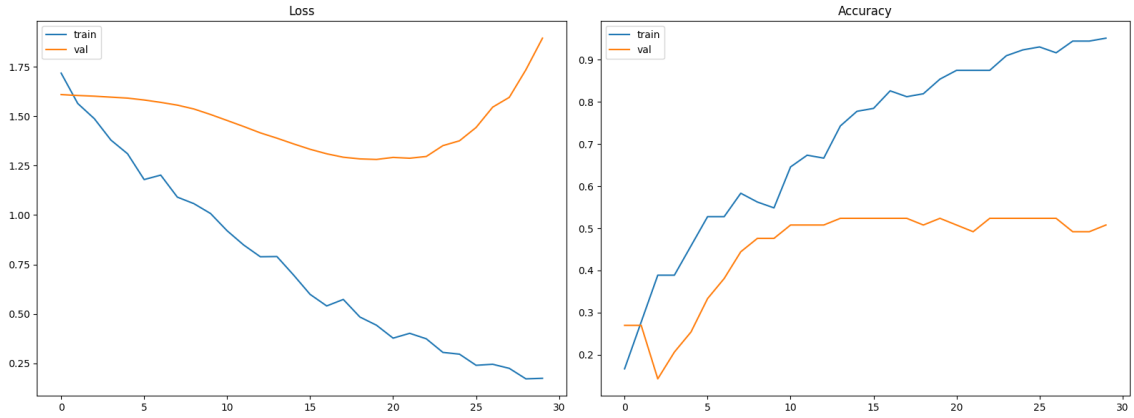


Figura 2.12: CNN + GRU Hybrid Model

Without Ridge regularization, the accuracy reaches 0.42. With Ridge, it improves slightly to 0.44.

Despite various hyperparameter tests, performance does not exceed the CNN-only model.

The hybrid is more stable but slightly less accurate. Combining CNN with LSTM results in even lower performance.

2.10 Related Work and Future Directions

Advanced architectures show promise for this task:

- **Long Expressive Memory (LEM)** networks, introduced at ICLR 2022, report accuracies above 90%.
- **Neural Rough Differential Equations** have also demonstrated strong results for long time series.

These models process full time series end-to-end, rather than relying on handcrafted features.

This concludes our study of the first classification task.

Capitolo 3

Feature Importance Analysis

Now, let us address the second task, which involves developing methods to identify which characteristics of the motion (values along the six "eigenworms") are most relevant to the outcome of our classification.

We have already analyzed classical classification using Random Forest and found that the autocorrelation feature of the second dimension was the most significant. Our focus now shifts to deep learning models, although these have yet to achieve high accuracy.

Interpreting neural networks is not straightforward, as it is generally challenging to understand how they allocate importance across input features. After reviewing the survey Interpretation of Time-Series Deep Models by Zhao et al., we estimate the importance of each dimension (in the vibration space) using several different approaches:

- **Backpropagation-based: Gradient-Based Sensitivity Analysis**

  The goal of this method is to compute the norm of the gradient with respect to each input dimension via backpropagation. We implemented two functions: one that records and quantifies the importance of each dimension based on the gradient values, and another that applies this analysis to a given model. For this purpose, we selected our best-performing deep learning models: the CNN and the Segmented LSTM.

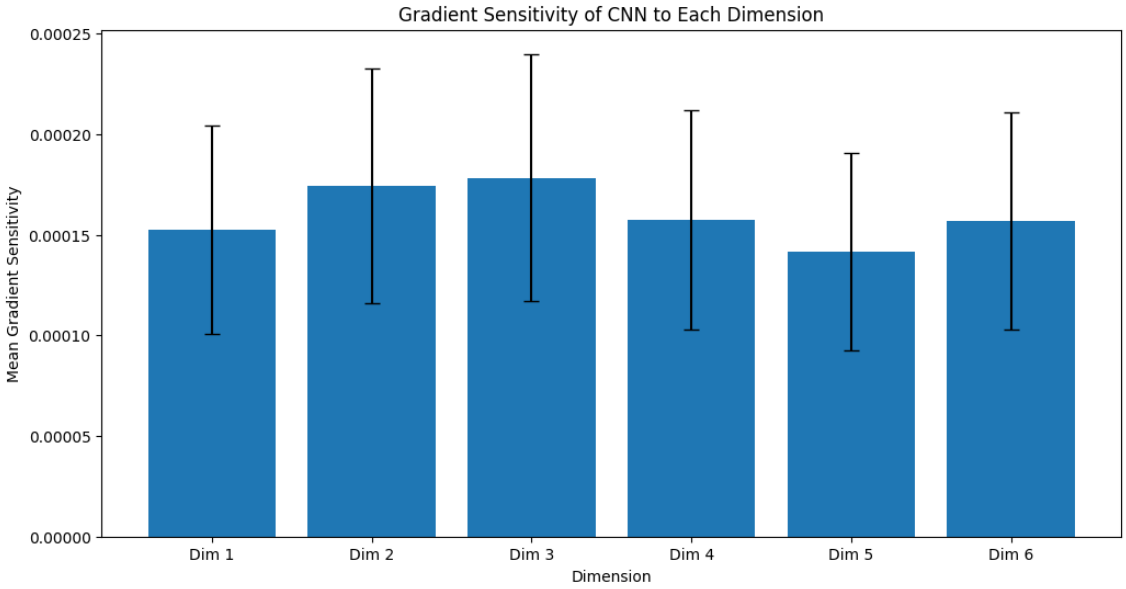


Figura 3.1: Gradient Sensitivity for the CNN Model

With the CNN model, we obtain a plot where all dimensions appear to carry some importance. However, dimensions 2 and 3 seem to be the most relevant. For the segmented model, the results are less conclusive. The issue likely stems from the model's limited accuracy, meaning that much of the analysis reflects somewhat random decisions.
Let us explore another method to see whether it yields more consistent results.

- **Perturbation-based: Permutation Importance**

  This approach evaluates the importance of each input feature by randomly permuting its values. Such perturbations disrupt the original temporal structure, allowing us to assess how crucial each dimension's structure is for the model's performance.

  For our CNN model, we obtain the following:

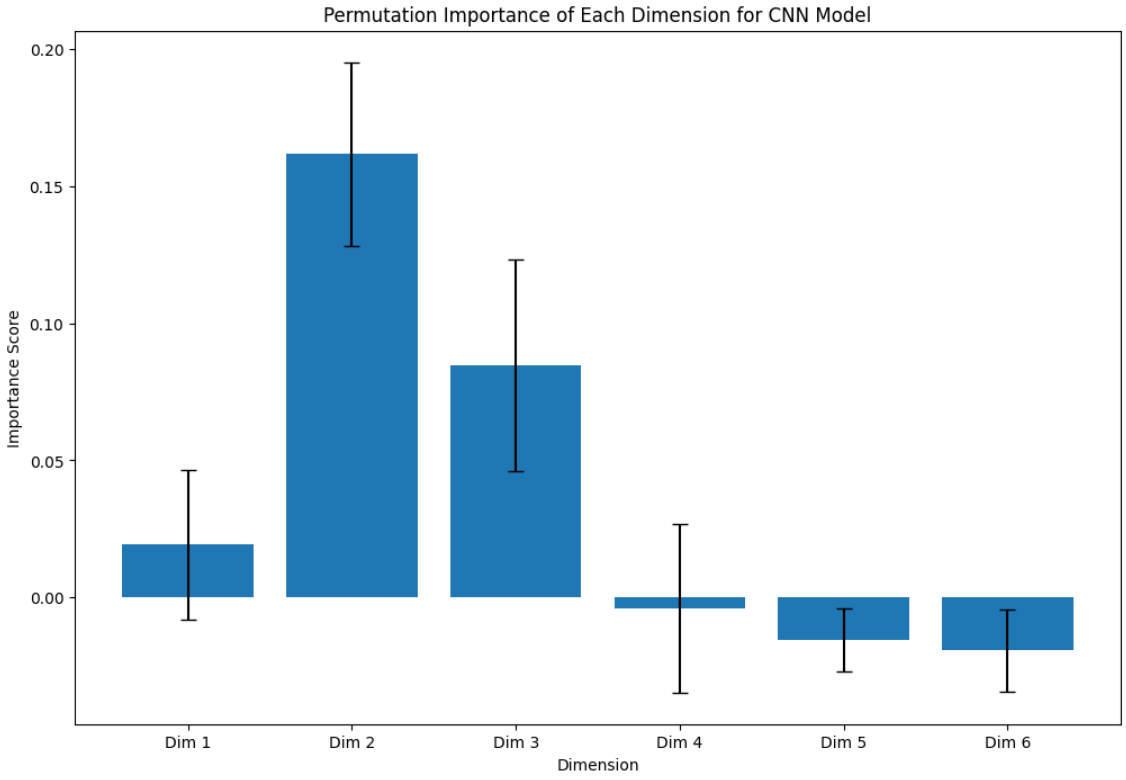


Figura 3.2: Permutation Importance for the CNN Model

Here, dimension 2 appears to have the greatest impact. Conversely, the last three dimensions contribute very little, suggesting they contain minimal relevant information for the CNN—indicating an absence of local patterns. This method proves to be more effective, as it highlights more distinct differences between the dimensions. Unlike the gradient-based approach, the resulting importance plot is not flat.

For the segmented LSTM model:

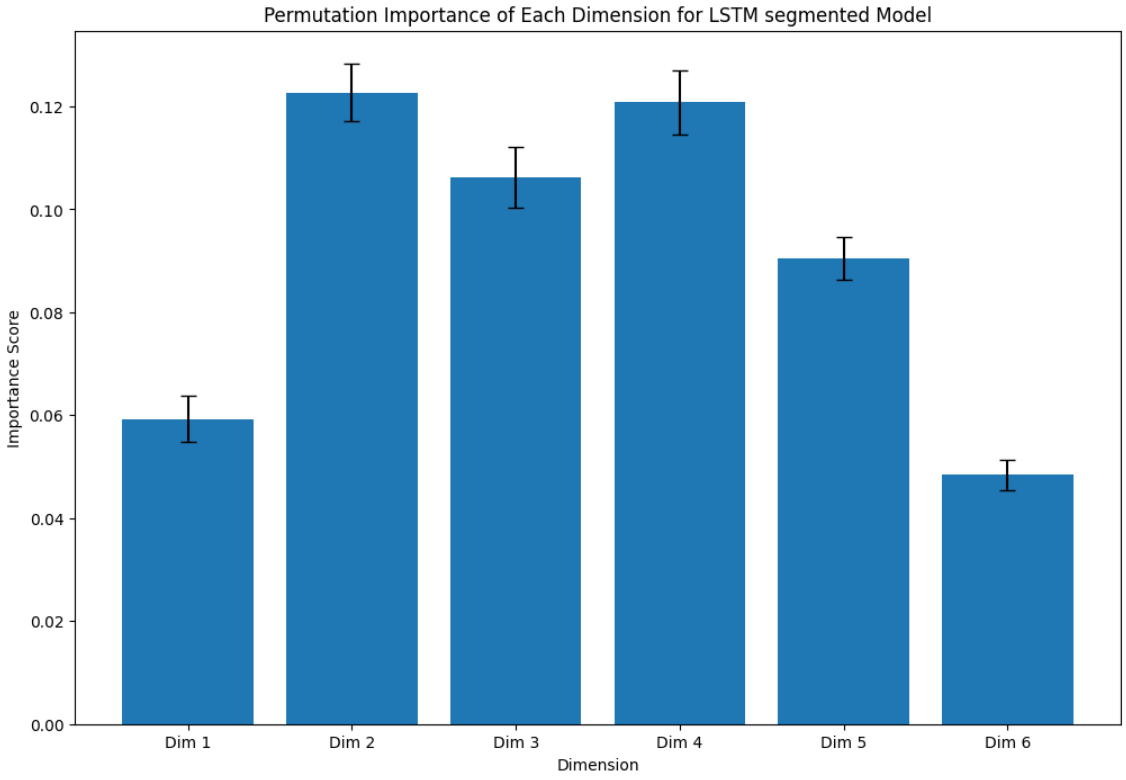


Figura 3.3: Permutation Importance for the Segmented LSTM Model

Here, the results are more mixed. Dimensions 2, 3, and 4 seem to be the most impactful.

Remark: It is expected that the histograms for the two models differ in shape, as they focus on different types of information within the data. From this analysis, we learn that dimensions 4, 5, and 6 contain little information about local patterns, as indicated by their low importance scores for the CNN. However, temporal information is still detected by the LSTM model, as shown in the final plot.

Given the class distribution is uneven—with one class representing nearly 40% of samples—our models are not very performant (hardly reaching 0.6 accuracy). The second task is partly biased by the poor performance of our models in the first task. It is thus difficult to assess which dimensions provide the most and best information when learning is poor.

Conclusion

In this work, we have explored various approaches for classifying time series data representing worm motion along six eigenworm dimensions. We began with classical feature engineering methods, followed by attempts to directly learn from raw multivariate time series using vanilla RNNs, LSTMs, GRUs, and hybrid architectures such as CNN+RNN models.

Our experiments revealed several challenges:

- The dataset is highly imbalanced, with one class representing nearly 40% of the samples.
- The raw time series are very long (17,984 time steps), which poses a challenge for recurrent models like RNNs and LSTMs that struggle to retain long-term dependencies. This necessitates aggressive downsampling or segmentation.
- Simple RNN and LSTM models tend to overfit while still achieving low test accuracy (around 0.27), which is worse than a naive classifier that always predicts the majority class.
- Segmenting the time series into shorter windows (200 time steps) and using majority voting across predictions significantly improves performance. With this strategy, we reached an accuracy of 0.61 — the second-best result among all tested models.
- CNN-based models, which focus on learning local temporal patterns, also achieved solid performance (accuracy between 0.58 and 0.60) without relying on recurrent layers. However, they are less stable and more prone to overfitting.
- Hybrid CNN+RNN architectures did not bring significant improvements over pure CNN models. In some cases, performance even deteriorated.
- Random Forest, using handcrafted statistical features — in particular the autocorrelation of the second dimension — outperformed all deep learning models, achieving the best accuracy of 92%.

For interpretability, we used gradient-based sensitivity analysis and permutation importance to estimate which eigenworm dimensions contribute most to classification. Both methods indicated that some dimensions (notably the third) have more impact, but these insights are limited by the modest classification performance.

Given more time, a promising direction would be to study and implement more advanced architectures designed for long time series, such as Long Expressive Memory (LEM) models or Neural Rough Differential Equations, which have shown state-of-the-art results in similar contexts.

Finally, the best-performing model was the Random Forest, outperforming all deep learning approaches. This success is largely attributed to the use of the autocorrelation feature, which appears to be highly discriminative.

Remark: Parts of this report — particularly the translation from French to English — were assisted by tools such as DeepL and large language models (LLMs).

Additional resource: The full implementation is available in the Colab notebook.